

Final Project Report

Facial Keypoint Detection

Course: Deep Learning
Universitat Pompeu Fabra

Team: **Omega**

Member 1 Name and NIA: **Pau Puig Guillén (285582)**

Member 2 Name and NIA: **Alejandro Poole Becerra (285987)**

Member 3 Name and NIA: **Joan Vicente Martín (284861)**

Date: 06/06/2026

1. Introduction

Facial keypoint detection is a computer vision task focused on locating relevant anatomical points on a face, such as the eyes, eyebrows, nose, and mouth. These landmarks are useful in several higher-level applications, including face tracking, facial expression analysis, biometric recognition, medical diagnosis support, and face alignment. In this project, we worked with the Kaggle Facial Keypoints Detection dataset, whose objective is to predict the position of facial landmarks in grayscale face images.

The dataset provides face images represented as 96 x 96 pixel intensity values, together with the corresponding coordinates of up to 15 facial keypoints. Each keypoint is represented by an x and y coordinate, giving a total of 30 possible target values per image. The training set contains labelled examples, while the test set contains only the face images. Some images in the training data contain missing annotations for certain keypoints, which makes the problem more realistic and requires careful handling during preprocessing and model training.

The task is formulated as a multi-output regression problem: given an input image, the model must predict continuous coordinate values for the facial landmarks. The official evaluation metric is Root Mean Squared Error (RMSE), which is suitable for this task because it penalizes large localization errors more strongly than metrics such as Mean Absolute Error. In this work, our objective was not only to train a predictive model, but also to build a clear experimental progression from a simple baseline to deep learning models developed from scratch. First, we implemented a mean-keypoint baseline that predicts the average position of each landmark. Then, we trained and compared several neural architectures, including a multilayer perceptron (MLP), a simple convolutional neural network (CNN), and more regularized CNN variants. The main goal was to determine whether models trained from scratch could improve the baseline and to analyse how architecture choices, loss functions, optimizers, regularization, and data augmentation affected performance.

2. State of the Art

Facial keypoint detection has evolved from simple coordinate regression methods to more advanced deep learning approaches that model facial geometry in a more structured way. In the simplest formulation, the model receives a face image and directly predicts the (x, y) coordinates of each landmark. This is still a useful approach for controlled datasets, especially when images are mostly frontal and landmarks are represented as a fixed set of points.

A common modern approach is heatmap regression. Instead of predicting each coordinate directly, the model produces one probability map for each landmark. In this map, the area indicates where the landmark is most likely to be located. The final coordinate can then be extracted from the peak of the heatmap. This strategy is popular because it preserves spatial information during prediction: the model does not only output a number, but learns an image-like representation of where each keypoint should be. For facial landmarks, this is especially useful because points such as eyes, mouth corners, and nose tip are strongly related to local visual structures.

Another relevant family of methods is based on anchor-based landmark localization. In these methods, the model starts from predefined reference face shapes, called anchors. These anchors represent typical landmark configurations. Instead of predicting all landmarks completely from zero, the model learns how to adapt or correct these reference shapes to match the face in the input image. AnchorFace is an example of this. Its main advantage is that it can use different reference templates for different face poses, which helps when the face is rotated or when the landmark layout changes significantly. Transfer learning is also common: models pretrained on larger face or landmark datasets can be reused as feature extractors and adapted to a smaller target dataset.

In practical applications, facial keypoint detection is usually not an isolated task. It is used as a component inside larger systems for face alignment, expression analysis, etc... This shows that modern landmark detection is important not only for predicting points accurately, but also for providing reliable facial geometry to downstream applications.

3. Methodology

3.1. Data Analysis

The dataset was downloaded from the Kaggle competition using kagglehub and then extracted into separate training and test folders. The main labelled file, training.csv, contains 7049 face images and 31 columns: 30 numerical columns corresponding to the x and y coordinates of 15 facial keypoints, plus one Image column. Each image is stored as a string of 9216 pixel values, which corresponds to a 96 x 96 grayscale image. During preprocessing, this string representation was converted into a NumPy array, reshaped to 96 x 96, and normalized to the [0, 1] range before being passed to the neural networks.

An initial exploratory analysis was performed to understand the structure and quality of the annotations. The dataset contains 15 facial landmarks, including points around the eyes, eyebrows, nose, and mouth. However, not all samples contain all 30 coordinate values. Missing annotations were a major characteristic of the dataset: only 2140 samples were complete, while 4909 samples had at least one missing coordinate. This means that only 30.36% of the available images contained all keypoints. The missingness pattern was not uniform: nose tip coordinates were available for all samples, eye centers had very few missing values, whereas several eyebrow, eye-corner, and mouth-corner coordinates were missing in more than two thirds of the dataset.

Split	Total samples	Complete samples	Incomplete samples	Complete %	Incomplete %
Full dataset	7049	2140	4909	30.36%	69.64%
Train	5639	1712	3927	30.36%	69.64%
Validation	1410	428	982	30.35%	69.65%

Table 1. Stratified train-validation split by keypoint annotation completeness.

Because of this imbalance between complete and incomplete annotations, the train-validation split was not created using a purely random partition. Instead, an auxiliary binary variable named *is_complete* was defined. This variable takes the value 1 when all 30 coordinate values are available and 0 otherwise. The final validation split was created using **stratification** over this variable. This ensured that the percentage of complete and incomplete samples remained almost identical in the full dataset, the training set, and the validation set. As shown in Table 1, the full dataset contains 30.36% complete samples, the training set contains 30.36%, and the validation set contains 30.35%.

This balanced split is important because the models are evaluated using only the available coordinates. If the validation set had contained a very different proportion of complete or incomplete samples, the resulting metrics would not have been directly comparable across experiments. The Kaggle test set was not used for metric computation because it does not contain ground-truth labels. Therefore, after the train-validation split, the training portion was further divided into a model-training subset and an internal test subset. This internal test set was used only for final evaluation after model selection, while the validation set was used for monitoring and selecting the best model during training.

Missing keypoints were handled using a masked learning strategy. In each dataset class, the target vector was built with 30 normalized coordinate values. Coordinates that were missing in the original dataframe were filled with the corresponding training-set mean only to maintain a fixed-size tensor. At the same time, a binary mask was stored to indicate which coordinates were actually available (1 for available coordinate, 0 otherwise). During loss computation and metric evaluation, this mask was used so that missing coordinates did not contribute to the result. This approach allowed the models to train on all available images instead of discarding almost 70% of the dataset.

3.2. Models and Optimization

The problem was formulated as a multi-output regression task. Each model receives a face image as input and returns 30 continuous values, corresponding to the x and y coordinates of the 15 facial keypoints. All target coordinates were normalized by dividing by 96 during training, and predictions were converted back to pixel coordinates when computing the final metrics. The experimental design included a non-neural baseline, a configurable multilayer perceptron, a configurable convolutional neural network, and a regularized CNN trained with photometric data augmentation.

Baseline model

The first model was a mean-keypoint baseline. It does not use image information. Instead, for each coordinate, the mean value was computed using only the training split while ignoring missing values. The same 30 mean values were then used as the prediction for every validation image. Although simple, this baseline is essential because it represents the performance of a model that only learns the average face geometry. All neural models were compared against this reference to verify whether they extracted useful visual information from the images.

MLP model

The MLP was implemented as a configurable fully connected neural network. Each 96 x 96 image was flattened into a vector of 9216 normalized pixel values. The model consisted of a sequence of linear layers, activation functions, optional dropout, and a final linear output layer with 30 neurons. The number and size of hidden layers were configurable, which allowed experiments with smaller, baseline, and larger architectures. ReLU was used as the default activation, although the implementation also allowed alternatives such as Tanh. Dropout was included after hidden layers as a regularization mechanism to reduce dependence on individual neurons. The main limitation of this model is that flattening the image removes the spatial structure of the face, making it harder to exploit local patterns such as eyes, nose, or mouth contours.

CNN models

The CNN models were designed to preserve and exploit the spatial structure of the images. In this case, the input tensor had shape 1 x 96 x 96, where the first dimension represents the grayscale channel. The configurable CNN class included two main presets: a simple configuration and a complex configuration.

The simple CNN used three convolutional blocks with 32, 64, and 128 channels followed by a dense regression head. Each block applied a 3 x 3 convolution, an activation function, max pooling, and optional dropout.

The complex CNN increased model capacity by adding a fourth convolutional block with 256 channels and a larger dense head with 512 and 256 units. The class also supported Batch Normalization, different dropout rates, and different activation functions.

The motivation for the CNN architecture is that facial keypoints are local and spatially structured. Convolutional filters can learn local features such as eye edges, eyebrow contours, mouth boundaries, and nose shape. Pooling layers reduce spatial resolution while keeping relevant features, and the final dense head maps the learned representation to continuous landmark coordinates. This makes CNNs more suitable than MLPs for image-based landmark localization.

Transfer Learning Model

A pretrained YKFK .h5 model, originally producing 68 landmarks with three values each, was loaded with a compatibility wrapper for older Keras DepthwiseConv2D configurations. FKD images were normalized, resized from 96 x 96 to 150 x 150, and repeated across three channels to match the RGB input expected by the pretrained MobileNetV2-based feature extractor. The original output head was replaced by a FKD regression head composed of Dense(512), Dropout(0.2), Dense(256), Dropout(0.2), and a final linear layer with 30 outputs. Training was performed in two stages: first, the feature extractor was frozen and only the new head was trained with Adam, masked MSE, learning rate 1e-3, batch size 64, and 20 epochs; then, the last 30 pretrained layers were fine-tuned with a lower learning rate of 1e-5 for another 20 epochs.

Model family	Input representation	Main configurable elements	Output
Mean baseline	No image input	Training-set coordinate means	30 coordinates
MLP	Flattened 9216-pixel vector	Hidden layers, activation, dropout	30 coordinates
Simple CNN	1 x 96 x 96 image tensor	Conv channels, dense layer, dropout, optimizer, loss	30 coordinates
Complex CNN	1 x 96 x 96 image tensor	Extra conv block, BatchNorm, dropout, weight decay, augmentation	30 coordinates
YKFK transfer learning	150 x 150 RGB image tensor	Frozen feature extractor, dense head, partial fine-tuning	30 coordinates

Table 2. Summary of the model families implemented in the project.

Loss functions and masking

Three masked loss functions were implemented: masked Mean Squared Error, masked L1 loss, and masked Smooth L1 loss. Each loss function was created with no internal reduction so that the coordinate-wise loss could be multiplied by the binary mask. The final loss was computed as the sum of valid coordinate losses divided by the number of available coordinates. This prevented missing labels from affecting the optimization. MSE was used as the most direct match to the RMSE evaluation metric, L1 was included as a more robust absolute-error alternative, and Smooth L1 was tested because it combines the stability of MSE for small errors with greater robustness to larger errors.

Optimizers and regularization

The experiments considered Adam, SGD with momentum, and RMSprop. Adam was used as the main optimizer because of its strong performance in deep learning problems and its adaptive learning-rate behaviour. SGD with momentum was included as a classical optimization baseline, while RMSprop was tested as an alternative adaptive optimizer. Learning rates were configurable and included values such as $1e-3$, $5e-4$, and $3e-4$ depending on the model family. Weight decay was used in regularized CNN experiments as L2 regularization, with values such as $1e-5$ or $1e-6$. Dropout was used in both MLP and CNN models, while Batch Normalization was mainly applied to complex CNN configurations to stabilize the distribution of intermediate activations.

Data augmentation was implemented only for the CNN experiments and only on the training split. The augmentation strategy was deliberately limited to photometric transformations: brightness adjustment, contrast adjustment, and Gaussian noise. These transformations simulate changes in illumination and image quality without modifying the geometry of the face. Therefore, unlike rotations, translations, or horizontal flips, they do not require transforming the keypoint coordinates. The validation and internal test sets were never augmented, which ensured that evaluation was performed on unmodified data.

Training loop and evaluation metrics

The training pipeline was written to be reusable across MLP and CNN models. Each batch contained an image tensor, a normalized target vector, and a mask. During each training epoch, the model produced 30 coordinate predictions, the masked loss was computed, gradients were backpropagated, and the optimizer updated the model parameters. After each epoch, the model was evaluated in inference mode on both the training and validation loaders. This is important because layers such as dropout behave differently during training and evaluation.

The main metric was RMSE in pixels, computed only over valid coordinates. This metric is aligned with the official Kaggle evaluation and strongly penalizes large localization errors. MAE (Mean Absolute Error) in pixels was also computed because it provides a more directly interpretable average absolute error. In addition, an NME (Normalized Mean Error) metric was defined for interpretability.

Overall, this methodology ensured that every model was trained and evaluated under comparable conditions. The use of a stratified split controlled the distribution of complete and incomplete annotations; the masked loss allowed incomplete samples to contribute useful information; configurable architectures enabled systematic experimentation; and the combination of RMSE, MAE, and NME provided both official and interpretable measures of model performance.

4. Experiments

4.1. MLP experiments

The first neural family was a multilayer perceptron trained on flattened 96×96 images. We tested a smaller MLP with hidden layers (256, 128), a standard MLP with hidden layers (512, 256, 128), and a larger version (1024, 512, 256) with stronger Dropout and weight decay. We also compared Adam and SGD optimizers and evaluated masked MSE against masked SmoothL1. These experiments were included to verify whether a dense network could improve the statistical baseline despite losing the spatial structure of the image.

4.2. Simple CNN experiments

The second family used simple convolutional neural networks. The reference configuration used convolutional channels (32, 64, 128), a dense layer of 256 units, ReLU activations and Dropout.

Additional variants included a smaller CNN with channels (16, 32, 64), a model with higher Dropout and weight decay, an SGD-based CNN, and a CNN trained with masked SmoothL1 loss. These experiments tested the advantage of preserving spatial image structure.

4.3. Regularized and augmented CNN experiments

The third family explored more complex CNNs with regularization and photometric data augmentation. These models used Batch Normalization, Dropout, weight decay, and training-only augmentation based on brightness, contrast and Gaussian noise (We did not apply augmentation on rotation or translation because for that we should modify the coordinate points values as well). We tested complex architectures with channels (32, 64, 128, 256) and dense layers (512, 256), medium versions with fewer convolutional blocks, Adam and RMSprop optimizers, MSE and SmoothL1 losses, and learning rates of $5e-4$ and $3e-4$.

4.4. Transfer learning from YKFK to FKD

The last experiment reused a pretrained YKFK .h5 model as a feature extractor for FKD. The original 68×3 landmark output was removed and replaced with a 30-coordinate regression head. Images were adapted to 150×150 RGB inputs, while labels and masks followed the same masked-learning protocol as the previous models. We first trained only the new dense head and then fine-tuned the last 30 pretrained layers using a much smaller learning rate. This tested whether facial representations learned from a related landmark dataset could transfer to the Kaggle FKD task.

5. Results

5.1. MLP Results

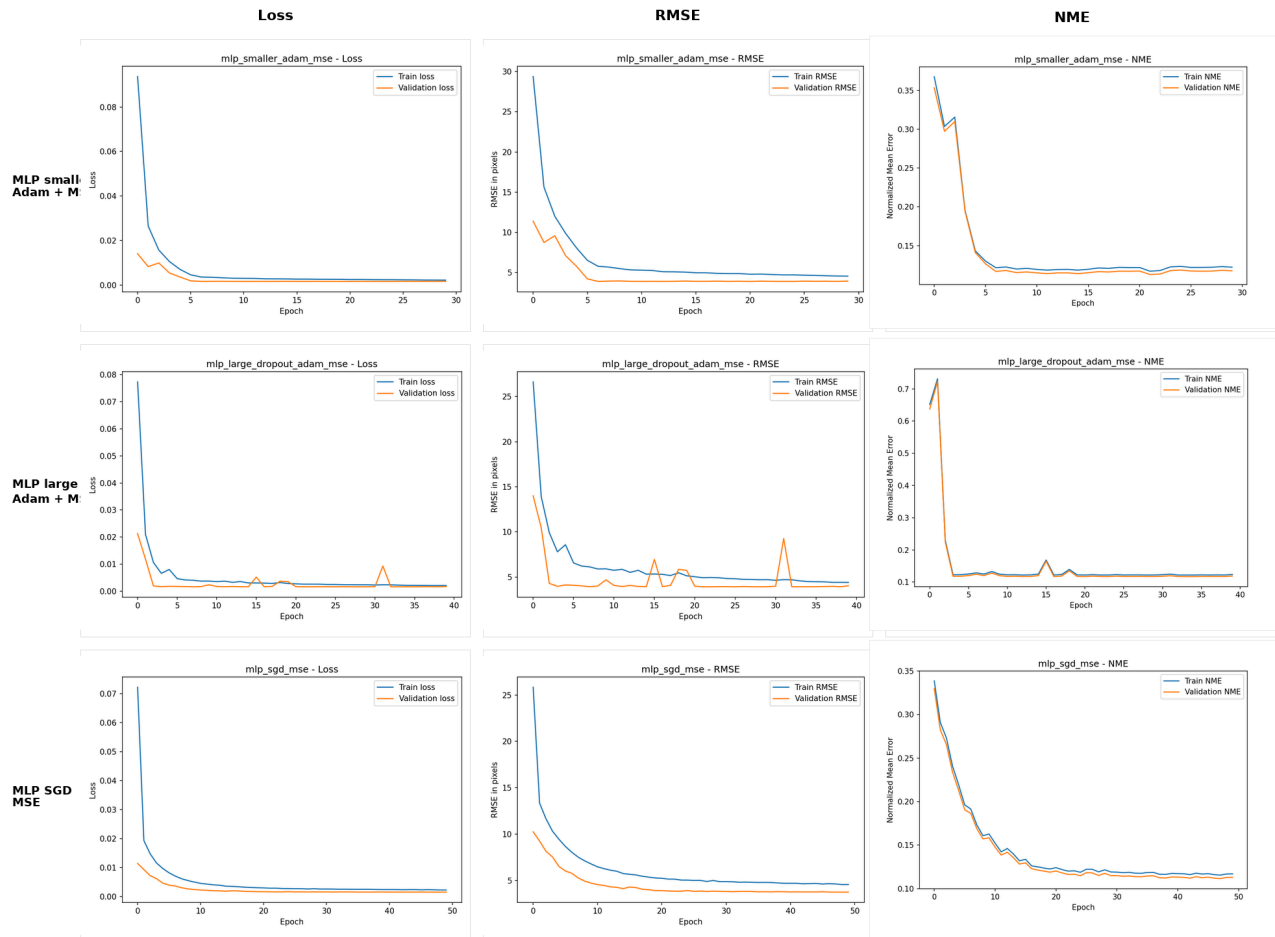


FIGURE 1. MLP training curves for three representative dense-network experiments. The rows compare a smaller Adam model, a larger model with stronger dropout, and the best MLP configuration based on SGD. The columns show loss (MSE), RMSE, and NME. All models show a strong early decrease in error, but the SGD configuration reaches the lowest validation RMSE among the MLP experiments.

Experiment	Configuration focus	Best Val RMSE	Test RMSE	Val Acc@5px
mlp_smaller_adam_mse	Smaller MLP, Adam, MSE	3.9124	3.8429	0.7137
mlp_large_dropout_adam_mse	Larger MLP, Adam, higher dropout	3.9137	3.8395	0.7145
mlp_sgd_mse	Baseline MLP, SGD with MSE	3.7478	3.6750	0.7428

TABLE 3. Main results for the selected MLP experiments. The table shows that the SGD-based MLP obtained the best result within the dense-model family. The Adam-based MLPs stayed close to the mean baseline, while the SGD configuration reduced both validation and internal test RMSE.

5.2. Simple CNN Results

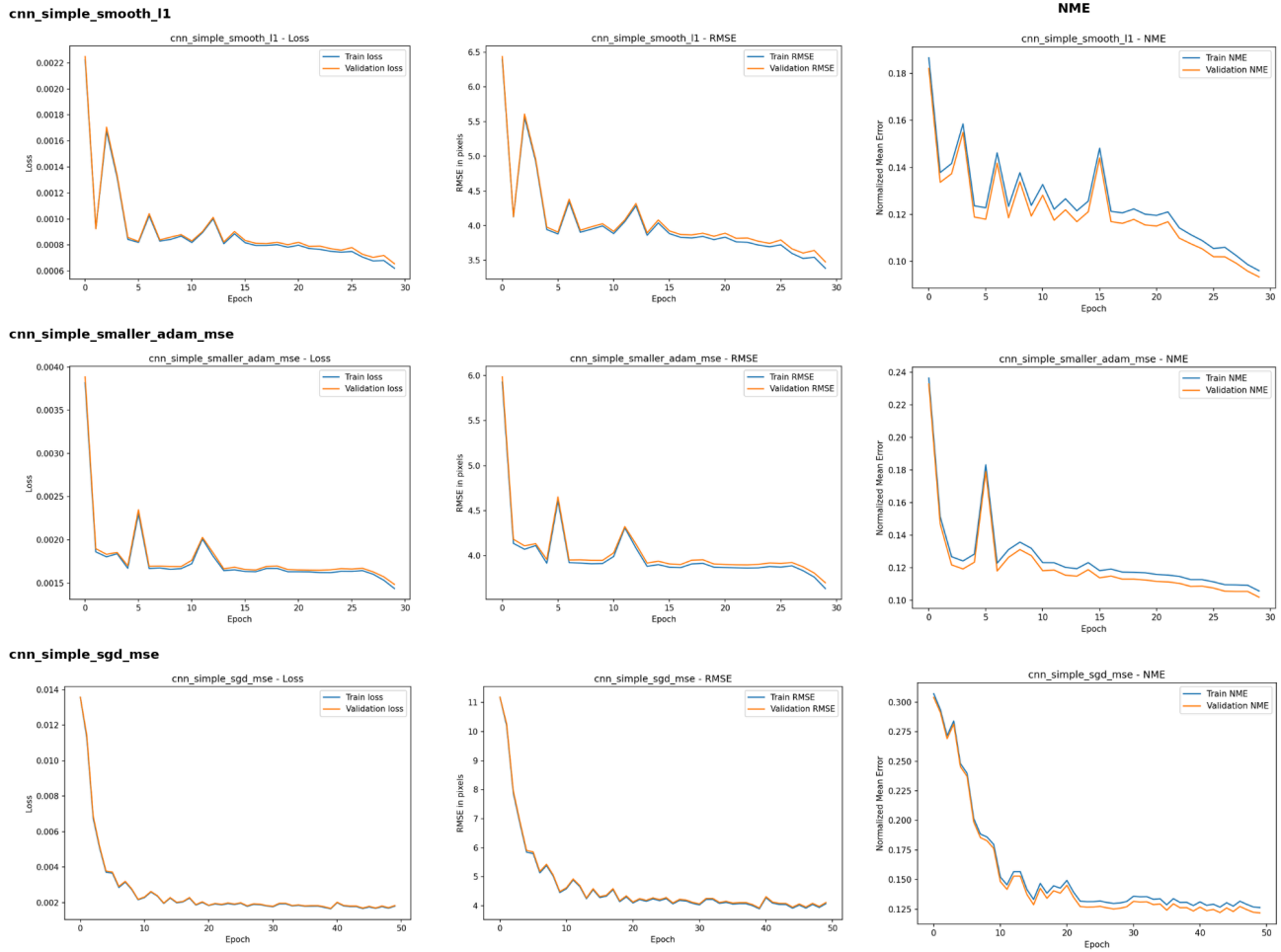


FIGURE 2. Training curves for the selected simple CNN experiments. The rows compare the Smooth L1 CNN, a smaller Adam CNN, and an SGD-based CNN. The plots show that the Smooth L1 configuration achieves the strongest final RMSE and NME, while the smaller Adam version remains competitive. The SGD CNN converges more slowly and finishes with higher RMSE than the Adam/Smooth L1 alternatives. The train and the validation in all the plots resulted in very similar results.

Experiment	Configuration focus	Best Val RMSE	Test RMSE	Val Acc@5px
cnn_simple_smooth_l1	Simple CNN, Smooth L1	3.4786	3.4002	0.7840
cnn_simple_smaller_adam_mse	Smaller CNN, Adam, MSE	3.6995	3.6219	0.7489
cnn_simple_sgd_mse	Simple CNN, SGD, MSE	3.9225	3.8366	0.7090

TABLE 4. Main results for the selected simple CNN experiments. The simple CNN family produced the strongest results of the project. The Smooth L1 experiment achieved the lowest validation and test RMSE, together with the highest NME. The smaller CNN also performed well, showing that a compact convolutional architecture can still extract useful spatial information. The SGD version was less effective in this setting.

5.3. Regularized and Augmented CNN Results

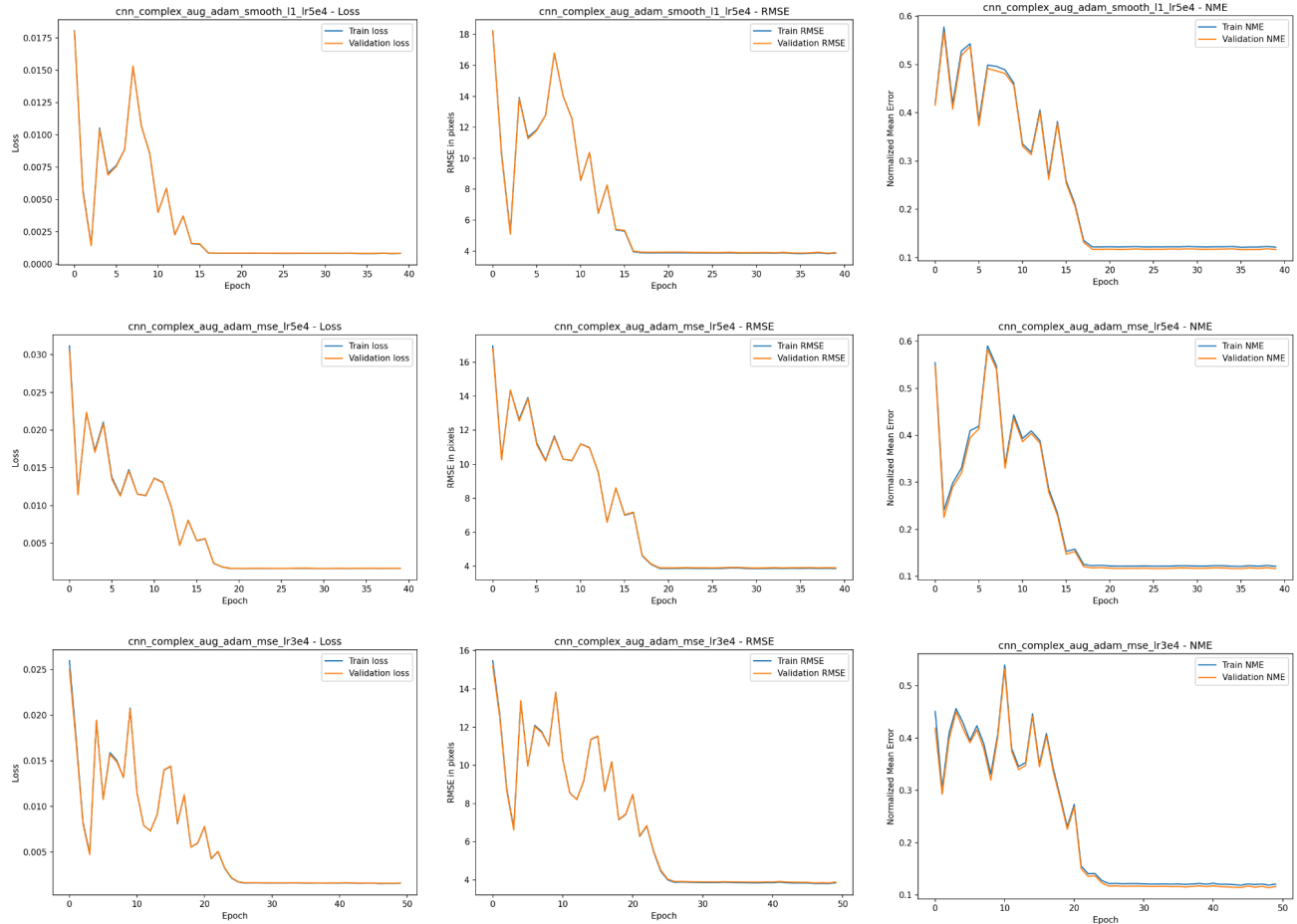


FIGURE 3. Training curves for the regularized and augmented CNN experiments. These models include Batch Normalization, Dropout, weight decay, and photometric data augmentation. The plots show a more unstable initial phase, followed by convergence near the baseline/simple-CNN range. The lower learning-rate MSE configuration is the best within this group, while Smooth L1 and the higher learning-rate MSE setting remain slightly weaker. All plots show that in the last epochs the loss decrease so fast and the accuracy increases so fast.

Experiment	Configuration focus	Best Val RMSE	Test RMSE	Val Acc@5px
cnn_complex_aug_adam_smooth_l1_lr5e4	Complex CNN, augmentation, Smooth L1	3.8791	3.8136	0.7186
cnn_complex_aug_adam_mse_lr5e4	Complex CNN, augmentation, MSE, lr=5e-4	3.8975	3.8282	0.7153

cnn_complex_aug_adam_mse_lr3e4	Complex CNN, MSE, lr=3e-4	3.8435	3.7761	0.7253
--------------------------------	---------------------------	--------	--------	--------

TABLE 5. Main results for the selected regularized and augmented CNN experiments. The regularized and augmented CNNs improved over the baseline in some cases, but did not outperform the best simple CNN. The best result in this family was obtained by reducing the learning rate to 3e-4. The plots and table show that the additional complexity and photometric augmentation produced stable final values, but the final RMSE remained higher than the Smooth L1 simple CNN.

5.4. Transfer Learning Results

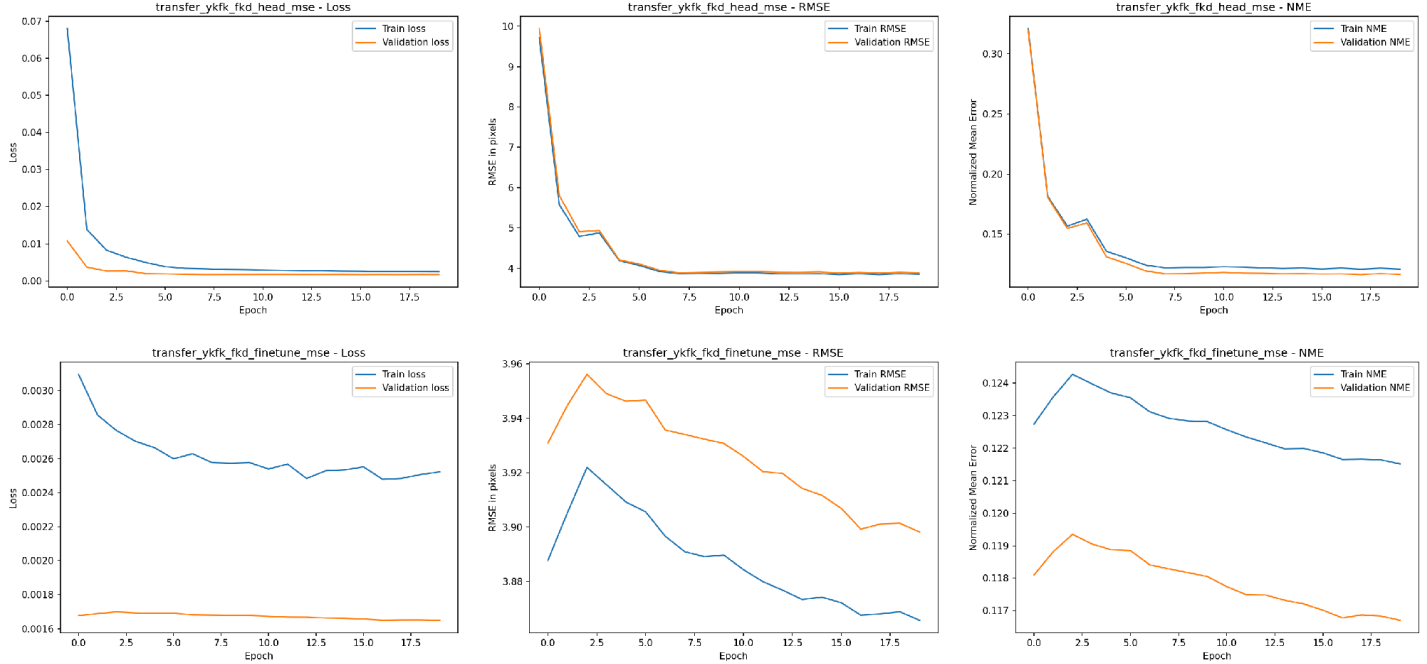


FIGURE 4. Transfer-learning curves for the frozen feature extractor and the fine-tuning stage. The first row shows the training of the new FKD head, where loss, RMSE, and NME drop quickly and then stabilize. The second row shows partial fine-tuning of the pretrained layers; validation RMSE and NME improve gradually, but remain close to the mean-keypoint baseline.

Baseline vs Transfer Learning Fine-tuning Results

experiment_name	rmse	normalized_mean_error	test_rmse	test_normalized_mean_error
mean_keypoint_baseline	3.907841	0.116925		
transfer_ykfk_fkd_finetune_mse	3.898177	0.116702	3.824378	0.119468

TABLE 6. Baseline versus transfer-learning fine-tuning results. Fine-tuning slightly improves the validation RMSE compared with the mean baseline, but the gain is small and the internal test RMSE remains above the best simple CNN result.

6. Discussion

The MLP experiments show that dense networks can learn useful information from image pixels, especially with the SGD-MSE configuration, but their improvement is limited because flattening 96 x 96 images removes the spatial relationships between facial regions.

The simple CNNs performed best overall. The Smooth L1 configuration achieved the lowest RMSE because it preserved local facial structure while being less sensitive to outlier coordinates than pure MSE. Close training and validation curves suggest good generalization rather than strong overfitting.

The regularized and augmented CNNs did not outperform the best simple CNN. Although Batch Normalization, Dropout, weight decay and photometric augmentation improved robustness in some runs, these models were harder to optimize and their final RMSE remained higher than the simpler Smooth L1 CNN.

The YKFK transfer-learning experiment produced only a marginal improvement over the mean-keypoint baseline: validation RMSE decreased from 3.9078 to 3.8982 and the normalized mean error from 0.1169 to 0.1167, while the test RMSE was 3.8244. The frozen feature extractor learned the new head quickly, but fine-tuning only improved the curves slowly and did not reach the simple CNN. This is probably because the pretrained model was trained for a different landmark format (68 landmarks with three values), while FKD uses 15 two-dimensional keypoints with many missing labels. In addition, the grayscale 96 x 96 FKD images had to be resized to 150 x 150 and repeated as RGB channels, which may have introduced a domain mismatch and limited the usefulness of the transferred features.

7. Conclusions

- The MLP improved the baseline only moderately because flattening the image removed important spatial structure.
- The simple CNN with Smooth L1 loss was the best model, achieving the strongest validation and test results.
- The complex regularized CNNs and the YKFK transfer-learning model did not outperform the best simple CNN.
- Future improvements should focus on geometric augmentation, landmark-specific losses, and more advanced transfer-learning or CNN architectures.